

Mid Semester Examination (27 March 2018)

High Performance Scientific Computing
(COMP3320/COMP6464)

Writing period : 90 Minutes duration

Study period : 15 Minutes duration

Permitted materials : NONE

Your answers MUST be written in the provided answer book(s). Make sure you have written your u-number on all answer books.

This exam is marked out of 100. You must answer all questions.

Your mark for this exam will contribute at most 20% of your total course mark, according to the marking scheme given on the course web page.

Student Number:

The following are used by your friendly examiner!

Q1 mark	Q2 mark	Q3 mark	Q4 mark	Q5 mark	Total mark

Question 1: Timing and profiling [20 total marks]

(a) When using a timer to measure the performance of your application code, it is important to know the “resolution” and “overhead” associated with that timer.

(i) Define timer resolution and timer overhead.

(ii) For the measurement of wall clock time and process time by system library calls, there is usually a large difference in the resolution and the overhead. Explain why this occurs and provide typical values for the resolution and overhead in each case.

(b) There are three different approaches to profile the performance of computer code. Name the approaches, briefly explain how they work, and give advantages and disadvantages for each.

Question 2: Floating point arithmetic [20 total marks]

(a) Modern processors support the extensive use of floating point numbers. Consider the normalized floating point system where the base $\beta=2$, the precision $t=3$, and the exponent range $[L:U] = [-2:2]$.

(i) Define what normalization of a floating point number is, and explain why it is beneficial.

(ii) Give the underflow limit of this floating point representation. [Note: you do not need to evaluate the values in decimal].

(iii) Explain how the gap between the underflow limit and zero can be reduced.

(b) The following code illustrates the effects of both approximation and rounding errors:

```
#include <stdio.h>
#include <math.h>
int main(void)
{ float h,x;
  int i;
  x=12.3456789;
  for (i=0, h=0.1; i<14; i++, h*=0.1)
    printf("%16.14f %8.1e %16.14f \n", 3.0*x*x, h,
          ((x+h)*(x+h)*(x+h))-(x*x*x)/h);
  return 0;
}
```

It computes the derivative x^3 both analytically and using a finite difference method:

$$\frac{dx^3}{dx} = 3x^2 \approx \frac{(x+h)^3 - x^3}{h}$$

Where h is the step size used in the finite difference. The code uses $x = 12.3456789$ and various step sizes. It gives the following output:

457.24739088661136	1.0e-01	460.96109472704148
457.24739088661136	1.0e-02	457.61786125682272
457.24739088661136	1.0e-03	457.28442892190822
457.24739088661136	1.0e-04	457.25109460003245
457.24739088661136	1.0e-05	457.24776125703750
457.24739088661136	1.0e-06	457.24742792359342
457.24739088661136	1.0e-07	457.24739458932828
457.24739088661136	1.0e-08	457.24739125320031
457.24739088661136	1.0e-09	457.24739092001755
457.24739088661136	1.0e-10	457.24739068066185
457.24739088661136	1.0e-11	457.24738147804351
457.24739088661136	1.0e-12	457.24718045061832
457.24739088661136	1.0e-13	457.24761214615535
457.24739088661136	1.0e-14	457.24539944595568

- (i) Define: (A) Rounding error; (B) Approximation error.
- (ii) Describe how this program illustrates both errors.
- (iii) For any given value of h , state how you would modify the code to:
 - (A) Decrease the effect of rounding errors;
 - (B) decrease the effect of approximation errors.
- (iv) For $h=10^{-7}$ write the expression for the relative error of the numerical derivative. [Note: You do not need to evaluate the expression].

Question 3: CPU Architecture [20 total marks]

- (a) Modern processors make extensive use of pipelining. Explain what pipelining is in the context of modern microprocessors.
- (b) Name three concepts other than pipelining that have been developed to improve processor performance and briefly describe how they work.
- (c) Pipeline throughput depends on the number of independent operations (N) and the pipeline depth (m)
 - (i) Define what pipeline depth means
 - (ii) What is the expected speedup versus a general purpose processing unit in the case of large N .
 - (iii) Describe two different situations which have a negative impact on pipeline performance, and briefly explain ways that modern hardware and compilers may circumvent these.

Question 4: Memory architecture [15 total marks]

(a) Sketch C code that illustrates

- (i) a cache friendly operation;
- (ii) a cache unfriendly operation.

In both cases provide a brief explanation as to why you consider the code to be cache friendly/unfriendly.

(b) Shown below is Fortran code to compute a matrix transpose. Does flipping the inner and outer loop increase performance? Give a detailed explanation of your answer.

```
do i = 1, N
  do j = 1, N
    A(j,i) = B(i,j)
  enddo
enddo
```

(c) Explain what the term cache thrashing means.

Question 5: Performance tuning [25 total marks]

(a) You are running on a machine with:

- Load/store latency of 2 cycles to L1 cache
- Floating point add or multiply latency of 3 cycles
- Superscalar architecture capable of 1 load/store, 2 floating point operations and 2 integer operations per clock cycle.

Consider the following loop in which all variables are independent (no aliasing):

```
for (i=0; i<2*N; i++)
  y[i] = y[i] + a * x[i]
```

- (i) What is the minimum number of clock cycles that it will take to execute one loop iteration, when the compiler has not perform loop unrolling and pipelining? Give an instruction schedule to explain your answer.
- (ii) What is the speedup achieved when the loop is unrolled 2 times? Include the new code and a new instruction schedule in your answer.

(b) Rewrite the following C code snippets to improve serial performance on an Intel i7 processor (similar to what you use on raijin) without changing code functionality. You can assume that variables are not aliased, and n is an even number.

(i) dot product

```
s = 0
for (i = 0; i < n; i++) {
    s += a[i]*b[i]
}
```

(ii) matrix transpose

```
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        a[j][i] = b[i][j]
    }
}
```

(c) Shown below are three functionally identical C functions to perform a matrix-matrix multiplication. For these functions also shown below are the anonymized benchmark results obtained on an Intel Sandybridge system (such as the raijin system used in this case). Assign which benchmark result (A, B or C) corresponds to which C function, and briefly explain the performance differences.

```
void matmuldot(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            for (k=0; k<n; k++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[j*n+k];
}
void matmuldax(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (i=0; i<n; i++)
        for (k=0; k<n; k++)
            for (j=0; j<n; j++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[k*n+j];
}
void matmulout(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (k=0; k<n; k++)
        for (i=0; i<n; i++)
            for (j=0; j<n; j++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[j*n+k];
}
```

Code	NSIZE	Time(sec)	MFLOPS

A	100	0.001110	1571.752319
B	100	0.000927	1882.578735
C	100	0.000946	1872.674683

A	1000	1.084468	1886.433838
B	1000	0.912717	2003.138062
C	1000	8.592217	1523.240845

A	2000	9.898019	1629.940552
B	2000	8.761745	1748.156616
C	2000	86.124147	1310.452026
